

StreamPet

Self-Hosted Twitch Stream Pet Setup & Configuration Guide

LLM: Ollama (GPU)

TTS: Piper (Local)

Bot: TwitchIO

UI: OBS Browser Source

RTX 3060 12 GB • Linux VM • OBS Studio

Revision 1.2 — May 2026

Table of Contents

1	System Overview	Architecture and component map
2	Prerequisites	What you need before starting
3	Installation	Step-by-step install walkthrough
4	Twitch Credentials	Registering an app and running get_token.py
5	Environment Configuration	All .env variables explained
6	LLM Configuration	Ollama models and GPU setup
7	TTS Configuration	Piper voices and audio routing
8	Pet Artwork	Swapping pet_idle.png / pet_talk.png
9	OBS Setup	VM IP, firewall, browser source, audio
10	SSH & Headless Operation	Token flow, tmux, SSH tunnelling
11	Running StreamPet	Persistent system service, management commands
12	Customisation	Personality, cooldowns, bubble style
13	Troubleshooting	Common issues and fixes
14	File Reference	Every file in the project explained

1. System Overview

StreamPet is a fully self-hosted Twitch chat bot and animated overlay. Viewers type **!ask <question>** in your Twitch channel; the pet responds with a spoken, animated reply generated entirely on your local machine — no cloud APIs, no subscriptions, no data leaving your VM.

Architecture

Component	Role	Technology
Twitch Bot	Listens for !ask, posts replies to chat	TwitchIO 2.x (pinned <3.0)
LLM Engine	Generates natural language answers	Ollama + llama3.2 (GPU)
TTS Engine	Converts text to speech WAV	Piper TTS binary + ONNX model
Web Server	Serves browser source + audio files	FastAPI + Uvicorn (Python)
WebSocket	Pushes events from bot to browser	FastAPI WebSocket endpoint
Browser Source	Renders pet, speech bubble, plays TTS	OBS Browser Source (HTML/JS)

Data Flow

Step	Action
1	Viewer types !ask What is gravity? in Twitch chat
2	TwitchIO bot receives the message, checks cooldown
3	Bot broadcasts thinking event via WebSocket — bubble appears
4	Bot sends question to Ollama REST API (localhost:11434)
5	Ollama runs llama3.2 (GPU-accelerated) and returns answer text
6	Answer piped to Piper TTS binary; WAV saved to static/audio/
7	Bot broadcasts speak event with text + audio URL via WebSocket
8	OBS Browser Source receives event, plays WAV, animates pet mouth
9	Bot also posts the text reply back to Twitch chat

2. Prerequisites

Before running the installer, confirm these items are in place:

Item	Requirement	Notes
OS	Linux (Ubuntu/Debian)	The installer uses apt-style tooling
GPU	NVIDIA Turing or newer (GTX 16xx / RTX 20xx+)	Minimum 4 GB VRAM. CUDA driver required for Ollama GPU acceleration. See GPU guide below.
Python	Python 3.10+	Python 3.14 confirmed working
python3-venv	apt package	sudo apt install python3.14-venv
curl	Command-line tool	Used by install.sh to download Piper and Ollama
OBS Studio	Any recent version	Browser Source plugin included by default
Twitch account	Bot account	Separate bot account recommended (free)
Internet	For initial setup only	Downloads Piper binary, voice model, and Ollama model

GPU Selection Guide

Ollama runs the LLM entirely on the GPU. The model you choose determines the minimum VRAM required. Piper TTS runs on CPU and uses no VRAM. All GPUs listed are Turing architecture (2018) or newer and support the CUDA versions required by Ollama.

Model	VRAM needed	Quality	Suggested GPUs
llama3.2 3B (default)	~3 GB	Good — fast responses, solid for chat	GTX 1660 Super (6 GB), RTX 2060 (6 GB), RTX 3050 (8 GB), RTX 3060 (12 GB), RTX 4060 (8 GB)
phi3:mini 3.8B	~3 GB	Good — very fast, optimised for instruction following	GTX 1660 Super (6 GB), RTX 2060 (6 GB), RTX 3050 (8 GB), RTX 4060 (8 GB)
llama3.2 7B mistral 7B	~5 GB	Better — more natural, creative answers	RTX 2060 Super (8 GB), RTX 2070 (8 GB), RTX 3060 (12 GB), RTX 3070 (8 GB), RTX 4060 Ti (8/16 GB)
llama3.1 8B gemma2 9B	~6 GB	Very good — noticeably higher quality	RTX 2070 Super (8 GB), RTX 2080 (8 GB), RTX 3060 (12 GB), RTX 3070 Ti (8 GB), RTX 4070 (12 GB)
llama3 70B (quantised)	~24 GB+	Excellent — overkill for stream chat	RTX 3090 (24 GB), RTX 4090 (24 GB), or dual-GPU setups

✓ Recommended sweet spot

For most streamers: RTX 3060 12 GB or RTX 4060 Ti 16 GB. Both run llama3.1 8B comfortably with headroom for a quantised 13B model. The extra VRAM also means the full model stays loaded between questions — no reload delay.

■ NVIDIA Driver Note

Ollama auto-detects your GPU via CUDA. If nvidia-smi is not in your PATH or returns no output, install the NVIDIA drivers before proceeding (prepare_vm.sh handles this automatically).

Check: nvidia-smi

Manual install guide: <https://docs.nvidia.com/cuda/cuda-installation-guide-linux/>

3. Installation

Follow these steps in order. The full process takes about 10–15 minutes on first run, mostly waiting for model downloads.

1 Prepare the VM (first time only — requires sudo)

Run the VM preparation script as root. It installs the NVIDIA driver, python3-venv, system tools, opens the firewall port, and reboots:

```
sudo bash prepare_vm.sh
```

After the reboot, SSH back in and verify:

```
nvidia-smi # GPU should appear
python3 -c 'import ensurepip; print("ok")' # should print ok
```

2 Run the installer

```
cd ~/streampet && ./install.sh
```

Creates a Python virtualenv, installs packages, downloads Piper TTS and voice model, installs Ollama, and pulls llama3.2 (~2 GB). Expected output:

```
[✓] pip ready
[✓] venv ready (.venv/)
[✓] Python packages installed
[✓] Piper binary downloaded
[✓] Voice model downloaded
[✓] Ollama installed
[✓] llama3.2 pulled
[!] .env created from .env.example — EDIT IT before starting!
```

3 Register your Twitch app and fill in .env

Register a developer app at dev.twitch.tv/console (see Section 4 for full details), then edit .env with the credentials:

```
nano .env
```

Set these five values — leave TWITCH_TOKEN blank for now (step 4 fills it in):

```
TWITCH_CLIENT_ID=your_client_id_from_dev_console
TWITCH_CLIENT_SECRET=your_client_secret_from_dev_console
TWITCH_USERNAME=your_bot_account_username
TWITCH_CHANNEL=your_stream_channel_name
```

4 Generate your Twitch token

Run the built-in OAuth helper. It detects SSH automatically and walks you through the authorisation flow, then writes TWITCH_TOKEN into .env:

```
python3 get_token.py
```

See Section 4 and Section 10.2 for detailed instructions including the SSH port-forwarding and paste-URL options.

5 Add your pet images

Drop two PNG files into the static/ folder:

- static/pet_idle.png — your pet with mouth closed
- static/pet_talk.png — your pet with mouth open

From your local machine: `scp pet_idle.png pet_talk.png user@vm-ip:~/streampet/static/`

Placeholder pink rectangles are generated automatically if you skip this step.

6 Install as a persistent system service

This makes StreamPet start automatically at every boot with no SSH session required:

```
sudo bash setup_service.sh
```

Or to run manually for testing:

```
./start.sh
```

7 Add OBS Browser Source

In OBS: Sources → + → Browser Source

- URL: `http://:8765` (not localhost — see Section 9)
- Enable: Control audio via OBS

4. Twitch Credentials

StreamPet needs a Twitch account to read chat. We strongly recommend creating a dedicated bot account (e.g. MyStreamPetBot) so your main account stays separate.

■ **twitchapps.com/tmi/ has been discontinued**

The third-party TMI token generator no longer works. StreamPet includes a built-in token helper (`get_token.py`) that handles the official Twitch OAuth flow entirely on your local machine — no third-party websites required.

4.1 Create a Bot Account (Recommended)

- Go to <https://www.twitch.tv> and sign out of your main account.
- Create a new Twitch account — e.g. YourChannelPet or StreamPetBot.
- This account does not need to stream; it is used only for chat.

4.2 Register a Twitch Developer Application

Twitch now requires every chat bot to be registered as a developer app. This is free and takes about two minutes.

- **1.** Log in to <https://dev.twitch.tv/console> as your BOT account (not your main).
- **2.** Click Register Your Application.
- **3.** Fill in the form:
 - Name: StreamPet (or any name you like)
 - OAuth Redirect URL: <http://localhost:17563> ← exactly as written
 - Category: Chat Bot
- **7.** Click Create, then click Manage on the app you just created.
- **8.** Copy the Client ID — paste it into `.env` as `TWITCH_CLIENT_ID=...`
- **9.** Click New Secret, copy it — paste into `.env` as `TWITCH_CLIENT_SECRET=...`

4.3 Generate the Token with `get_token.py`

Once Client ID and Client Secret are in `.env`, run the included helper script. It uses only Python standard library — no extra packages needed.

```
python3 get_token.py
```

The script will:

- Open your browser to the official Twitch OAuth authorisation page.
- Ask you to log in as your BOT account and click Authorise.
- Catch the redirect on <http://localhost:17563> (no internet exposure).

- Exchange the code for a User Access Token via the Twitch API.
- Write `TWITCH_TOKEN=oauth:...` directly into your `.env` file.

■ Keep your credentials secret

Never commit `.env` to Git or share it. The Client Secret and access token are equivalent to passwords. If compromised, delete the app at dev.twitch.tv/console and create a new one, then re-run `get_token.py`.

4.4 Mod the Bot in Your Channel (Recommended)

Modding the bot prevents Twitch rate-limiting its chat messages. In your channel chat, type:

```
/mod YourBotUsername
```

4.5 .env Values for Twitch

Variable	Example	Description
<code>TWITCH_CLIENT_ID</code>	<code>abc123def456...</code>	App Client ID from dev.twitch.tv/console
<code>TWITCH_CLIENT_SECRET</code>	<code>xyz789...</code>	App Client Secret (keep private)
<code>TWITCH_TOKEN</code>	<code>oauth:abc123...</code>	Written automatically by <code>get_token.py</code>
<code>TWITCH_USERNAME</code>	<code>StreamPetBot</code>	Bot account Twitch username (lowercase)
<code>TWITCH_CHANNEL</code>	<code>yourchannel</code>	Your stream channel name (no #, lowercase)

5. Environment Configuration (.env)

All runtime settings live in the .env file in the project root. Copy .env.example to .env and edit it. The table below documents every available variable.

Twitch

Variable	Default	Description
TWITCH_CLIENT_ID	abc123...	App Client ID from dev.twitch.tv/console
TWITCH_CLIENT_SECRET	xyz789...	App Client Secret from dev.twitch.tv/console
TWITCH_TOKEN	oauth:...	User Access Token — written by get_token.py
TWITCH_USERNAME	StreamPetBot	Bot Twitch username (lowercase)
TWITCH_CHANNEL	yourchannel	Your channel name, no # sign

LLM — Ollama

Variable	Default	Description
OLLAMA_URL	http://localhost:11434	Ollama API base URL
OLLAMA_MODEL	llama3.2	Model name (must be pulled via ollama pull)
MAX_TOKENS	150	Max reply length in tokens (~75 words)

Pet Personality

Variable	Default	Description
PET_NAME	Pixel	Pet's name used in LLM system prompt and chat
PET_PERSONALITY	...	Full personality prompt injected into every LLM call

TTS — Piper

Variable	Default	Description
PIPER_BINARY	./piper/piper	Path to Piper executable
PIPER_MODEL	./piper/en_US-amy-medium.onnx	Path to .onnx voice model

Server

Variable	Default	Description
SERVER_HOST	0.0.0.0	Bind address (0.0.0.0 = all interfaces)
SERVER_PORT	8765	HTTP port for browser source and WebSocket

Behaviour

Variable	Default	Description
COOLDOWN_SECONDS	15	Per-user cooldown between !ask commands

6. LLM Configuration (Ollama)

StreamPet uses Ollama to run large language models locally. The RTX 3060 12 GB VRAM is enough for 7B parameter models like llama3.2, mistral, or gemma2 at full GPU speed.

6.1 Checking Ollama

```
ollama list # see pulled models
ollama ps # see what is currently loaded
curl http://localhost:11434/api/tags # confirm API is up
```

6.2 Pulling Alternative Models

Model	VRAM	Quality	Pull command
llama3.2 (default)	~4 GB	Excellent	ollama pull llama3.2
mistral	~4 GB	Very good	ollama pull mistral
gemma2:9b	~6 GB	Excellent	ollama pull gemma2:9b
llama3.1:8b	~5 GB	Excellent	ollama pull llama3.1:8b
phi3:mini	~2.5 GB	Good / fast	ollama pull phi3:mini

After pulling a new model, update OLLAMA_MODEL in .env and restart.

6.3 Customising the Personality Prompt

The system prompt is built in llm.py from the PET_NAME and PET_PERSONALITY env vars. To change it, edit .env:

```
PET_NAME=Blaze
PET_PERSONALITY=You are a sarcastic but loveable gamer cat. Keep answers under 2 sentences.
```

■ Response Length Tip

Keep MAX_TOKENS between 80–200 for stream use. Very long replies make TTS slow and viewers lose interest. The default of 150 (~75 words) is a good balance.

7. TTS Configuration (Piper)

Piper TTS is a fast, self-hosted neural text-to-speech system. It runs as a standalone binary and requires no GPU — leaving your RTX 3060 fully available to Ollama. Audio is generated in real-time (typically under 1 second for short replies).

7.1 Voice Models

Piper voices are distributed as .onnx files paired with a .onnx.json config. The default voice installed is en_US-amy-medium. To change voices:

Step 1 — Browse available voices at:

```
https://huggingface.co/rhasspy/piper-voices
```

Step 2 — Download the .onnx and .onnx.json files into the piper/ folder:

```
cd ~/streampet/piper
curl -L https://huggingface.co/rhasspy/piper-voices/resolve/main/
en/en_US/lessac/medium/en_US-lessac-medium.onnx -o en_US-lessac-medium.onnx
curl -L https://huggingface.co/rhasspy/piper-voices/resolve/main/
en/en_US/lessac/medium/en_US-lessac-medium.onnx.json -o
en_US-lessac-medium.onnx.json
```

Step 3 — Update .env:

```
PIPER_MODEL=./piper/en_US-lessac-medium.onnx
```

7.2 Popular Voice Options

Voice ID	Gender	Quality	Notes
en_US-amy-medium	F	Medium	Default — fast, clear
en_US-lessac-medium	M	Medium	Natural male voice
en_US-libritts-high	F/M	High	Best quality, slightly slower
en_GB-alba-medium	F	Medium	British English accent

7.3 Testing TTS Manually

```
echo "Hello from StreamPet!" | ./piper/piper \
--model ./piper/en_US-amy-medium.onnx \
--output_file /tmp/test.wav

# Play it back
aplay /tmp/test.wav # or: paplay, mpv, ffplay
```

■ Audio Cleanup

Generated WAV files are automatically deleted after 5 minutes (300 seconds). This prevents the static/audio/ folder from filling up disk space during long streams. The cleanup interval is hardcoded in tts.py and can be changed there.

8. Pet Artwork

StreamPet uses two PNG image files to animate the pet. The browser source swaps between them at 150 ms intervals while TTS audio plays, creating a talking animation.

8.1 Required Files

File	State	When used
static/pet_idle.png	Mouth closed / default pose	All times except when speaking
static/pet_talk.png	Mouth open	Alternates with idle during TTS playback

8.2 Artwork Recommendations

- PNG with transparent background works best as an OBS overlay.
- Recommended canvas size: 300 x 400 px or 400 x 500 px.
- The pet image width is set to 300px in CSS — edit static/style.css line image-rendering to change.
- Keep both images the same canvas size so the swap is seamless.
- You can use pixel art, illustrated characters, or even a VTuber-style chibi.
- Tools: Aseprite (pixel art), GIMP, Krita, or Procreate (export as PNG).

8.3 Changing Image Size in OBS

By default the image renders at 300px wide. To override this via OBS Custom CSS in the Browser Source properties:

```
#pet-img {  
width: 400px; /* change to your preferred size */  
}  
#scene {  
width: 500px; /* scene container width */  
height: 500px;  
}
```

8.4 Talk Frame Speed

The mouth alternates every 150 ms by default. To change this, edit static/pet.js:

```
const TALK_FRAME_MS = 150; // line 17 – change to 100 for faster
```

✓ Placeholder Images

run `make_placeholders.py` to regenerate the solid-colour placeholder PNGs at any time. These are safe to delete once you have your own artwork in place.

9. OBS Setup

StreamPet displays as a Browser Source in OBS Studio. The browser source both renders the pet animation and plays the TTS audio through OBS's audio mixer. Because the Linux VM is headless, OBS runs on your streaming PC and connects to the VM over your local network — not via localhost.

9.1 Find the VM's LAN IP Address

Run this on the VM (over SSH) before setting up OBS:

```
hostname -I | awk '{print $1}'  
# example output: 192.168.1.42
```

This is the IP address you will use in the OBS Browser Source URL. For a stable setup, assign a static IP or DHCP reservation to the VM in your router.

9.2 Open Port 8765 on the VM Firewall

If the VM has a firewall active (ufw), allow the StreamPet port:

```
sudo ufw allow 8765/tcp  
sudo ufw status # confirm rule is listed
```

Verify StreamPet is reachable from your streaming PC:

```
curl http://:8765/health
```

9.3 Adding the Browser Source in OBS

- 1. In OBS, click the + button in the Sources panel.
- 2. Select Browser from the source type list.
- 3. Name it StreamPet (or any name you prefer).
- 4. In the URL field enter: `http://:8765` (use actual IP, not localhost)
- 5. Set Width and Height to match your pet PNG canvas size (e.g. 420 x 420).
- 6. Check the box: Control audio via OBS — this is required for TTS to play.
- 7. Leave all other settings at their defaults and click OK.
- 8. Drag and reposition the source in your scene as needed.

■ Use VM IP, not localhost

localhost in the Browser Source URL refers to your streaming PC, not the VM. The browser source will load a blank page if you use localhost. Always use the VM's actual LAN IP address, e.g. `http://192.168.1.42:8765`

■ Control audio via OBS — Required

Without this checkbox the browser source runs in a sandboxed audio context that OBS cannot capture. TTS audio will not be heard on stream. Always enable this option.

9.4 Audio Routing

Once Control audio via OBS is enabled, a new audio track called Browser appears in the OBS Audio Mixer. You can:

- Adjust pet volume independently of your mic or desktop audio.
- Add filters (e.g. compressor, noise gate) to the pet audio track.
- Route pet audio to a specific output track for recording vs. streaming.

9.5 Transparent Background

The browser source has a transparent background by default. This lets the pet float over your scene. Do not add a background colour in OBS Browser Source settings — leave it at default.

9.6 Recommended OBS Scene Setup

Source	Type	Notes
StreamPet	Browser	http://192.168.x.x:8765 — Control audio via OBS enabled
Webcam	Video Capture	Place behind or beside the pet
Game Capture	Game/Window Capture	Bottom layer
Chat Overlay	Browser	Optional — shows alongside the pet

10. SSH & Headless Operation

The Linux VM is headless — it has no monitor, keyboard, or desktop environment. All day-to-day operation is done over SSH from your Windows or Mac machine. This section covers every situation where the headless environment requires special handling.

10.1 What Works Normally Over SSH

Component	SSH compatible?	Notes
install.sh	Yes	Runs entirely in the terminal
start.sh / main.py	Yes	Terminal process, no display needed
Ollama daemon	Yes	Background service, no display needed
Piper TTS	Yes	Subprocess, writes WAV to disk
Twitch bot	Yes	Network connection to Twitch only
FastAPI server	Yes	Binds 0.0.0.0 — reachable from OBS over LAN
get_token.py	Yes — see below	Auto-detects SSH; uses paste or tunnel method

10.2 Getting the Twitch Token Over SSH

get_token.py detects SSH sessions automatically and switches to headless mode. It offers two options:

Option A — SSH Port Forwarding (Recommended)

Open a second terminal on your LOCAL machine (not the VM) and run:

```
ssh -L 17563:localhost:17563 your-user@192.168.x.x
# Keep this terminal open
```

Then run get_token.py on the VM. Copy the URL it prints, open it in your local browser, and log in as your bot account. Twitch redirects to http://localhost:17563 — the SSH tunnel forwards that request to the VM. The token is captured and saved automatically.

Option B — Paste the Redirect URL (No Extra Setup)

If you cannot or do not want to use port forwarding:

1. Run python3 get_token.py on the VM — copy the auth URL it prints.
2. Open that URL in any browser on any machine.
3. Log in as your bot account and click Authorise.
4. Your browser will try to open http://localhost:17563/?code=... and show an error — this is expected.
5. Copy the full URL from your browser's address bar (the one showing the error).
6. Paste it into the get_token.py terminal prompt and press Enter.

- 7. The token is extracted, exchanged, and saved to .env automatically.

10.3 Keeping StreamPet Running After SSH Disconnect

If you start StreamPet in a plain SSH terminal, it will stop when you disconnect. The recommended approach is the system-level systemd service installed by setup_service.sh.

Recommended: system service (starts at every boot, no login needed)

```
sudo bash setup_service.sh

# Management:
sudo systemctl status streampet
sudo journalctl -u streampet -f # live logs over SSH
sudo systemctl restart streampet # after .env changes
sudo systemctl stop streampet
```

This is a system-level service (/etc/systemd/system/streampet.service), not a user service. It starts during boot before any login — no SSH session required.

Alternative: tmux (useful during development/testing)

```
tmux new -s streampet # create a named session
./start.sh # start inside tmux
# Ctrl+B then D to detach — StreamPet keeps running

tmux attach -t streampet # reattach later
```

■ System service vs user service — which to use

setup_service.sh installs a system-level service that starts at boot with no login. A user-level service (systemctl --user) only starts when you SSH in — avoid it for production. Use tmux only while actively developing or debugging.

11. Running StreamPet

StreamPet is designed to run as a persistent system service — it starts automatically at every boot and restarts itself if it crashes, with no SSH session or manual intervention required.

✓ Recommended: always use the system service

The systemd system service is the correct way to run StreamPet in production. It starts before any user logs in, survives SSH disconnects, and recovers from crashes. Manual start via `start.sh` is only for development and testing.

11.1 Install the System Service (Once)

After completing `install.sh`, `get_token.py`, and dropping in your pet artwork, run the service installer script. This requires `sudo` once only:

```
sudo bash setup_service.sh
```

The script will:

- Write `/etc/systemd/system/streampet.service` (system-level, not user-level).
- Enable the service so it starts automatically at every boot.
- Start it immediately — StreamPet is live within seconds.
- Print the OBS Browser Source URL with the VM's actual LAN IP.
- Run a pre-flight check: refuses to install if `TWITCH_TOKEN` or `TWITCH_CHANNEL` are still placeholders.

■ System service vs user service — why it matters

A user-level service (`systemctl --user`) only starts when the user logs in. On a headless VM this means it never starts unless you SSH in. A system-level service (`/etc/systemd/system/`) starts during boot before any login — which is what we want.

11.2 Day-to-Day Service Management

Task	Command
Check status	<code>sudo systemctl status streampet</code>
Follow live logs	<code>sudo journalctl -u streampet -f</code>
Restart (e.g. after <code>.env</code> edit)	<code>sudo systemctl restart streampet</code>
Stop	<code>sudo systemctl stop streampet</code>
Start (if stopped)	<code>sudo systemctl start streampet</code>

Disable autostart	<code>sudo systemctl disable streampet</code>
Remove service entirely	<code>sudo bash remove_service.sh</code>

11.3 Updating Configuration

Any time you edit `.env` (change model, personality, cooldown, etc.), restart the service to apply the change:

```
nano ~/streampet/.env
sudo systemctl restart streampet
sudo journalctl -u streampet -f # confirm it started cleanly
```

11.4 What the Service Unit Looks Like

The installed unit file at `/etc/systemd/system/streampet.service`:

```
[Unit]
Description=StreamPet – Self-Hosted Twitch Stream Pet
After=network-online.target ollama.service
Wants=network-online.target ollama.service

[Service]
Type=simple
User=
WorkingDirectory=/home//streampet
EnvironmentFile=/home//streampet/.env
ExecStart=/home//streampet/.venv/bin/python3 main.py
Restart=always
RestartSec=10
StartLimitBurst=5
StandardOutput=journal

[Install]
WantedBy=multi-user.target
```

11.5 Manual Start (Development Only)

If the system service is running, `start.sh` will detect it and show service status instead of launching a second instance. To run manually for debugging, stop the service first:

```
sudo systemctl stop streampet
cd ~/streampet && ./start.sh

# Re-enable the service when done:
sudo systemctl start streampet
```

11.6 Expected Log Output

```
$ sudo journalctl -u streampet -f
```

```
[*] Starting StreamPet
Pet name : Pixel
Channel : #yourchannel
LLM model : llama3.2
INFO: Uvicorn running on http://0.0.0.0:8765
[Bot] Connected as StreamPetBot | Channel: #yourchannel
[WS] Client connected (1 total)
```

12. Customisation

12.1 Personality and Prompt

The LLM system prompt is assembled in `llm.py` from your `.env` values. To give your pet a distinct voice, edit `PET_PERSONALITY` in `.env`. Examples:

```
# Snarky gamer cat
PET_PERSONALITY=You are a sarcastic gamer cat who loves roasting viewers (gently).
Keep it under 2 sentences.

# Wise wizard owl
PET_PERSONALITY=You are an ancient wise owl who speaks in riddles and metaphors.
One sentence answers only.

# Cheerful corgi
PET_PERSONALITY=You are an enthusiastic golden corgi who LOVES everything. Bark
once in every reply.
```

12.2 Cooldown

`COOLDOWN_SECONDS` in `.env` controls how long each viewer must wait between `!ask` uses. Set it lower (5–10s) for very active chats, higher (30–60s) for slower ones. The default is 15 seconds.

12.3 Speech Bubble Style

Edit `static/style.css` to change bubble appearance:

```
#bubble {
border: 3px solid #f472b6; /* border colour */
border-radius: 20px; /* roundness */
background: rgba(255,255,255,0.95);
box-shadow: 0 4px 20px rgba(244,114,182,0.45);
}
#bubble-username { color: #db2777; } /* username colour */
#bubble-text { font-size: 14px; } /* text size */
```

12.4 Bubble Linger Time

After TTS finishes, the speech bubble stays visible for 4 seconds before fading. Change this in `static/pet.js`:

```
const BUBBLE_LINGER_MS = 4000; // line 18
```

12.5 Adding More Commands

To add new bot commands, edit bot.py and add methods decorated with `@commands.command()`.
Example:

```
@commands.command(name='pet')
async def pet_command(self, ctx: commands.Context):
    await ctx.send(f'{cfg.PET_NAME} purrs contentedly* uwu')
```

13. Troubleshooting

Service fails to start / crashes at boot

- Check the journal for error details: `sudo journalctl -u streampet -xe`
- Most common cause: TWITCH_TOKEN expired. Re-run `python3 get_token.py`, then `sudo systemctl restart streampet`.
- Check Ollama is running: `sudo systemctl status ollama`
- Verify .env paths: PIPER_BINARY and PIPER_MODEL must be absolute or relative to your streampet/ directory.
- If the venv was deleted, re-run `./install.sh` then `sudo bash setup_service.sh` again.

OBS Browser Source shows blank / cannot connect

- This usually means localhost was used in the URL instead of the VM's LAN IP.
- Fix: right-click the source → Properties → change URL to `http://192.168.x.x:8765`
- Find the VM IP: run `hostname -I | awk '{print $1}'` on the VM over SSH.
- Confirm StreamPet is running: `curl http://:8765/health`
- If curl fails from another machine, open port 8765: `sudo ufw allow 8765/tcp`

get_token.py — browser won't open / no callback

- On a headless VM this is expected — the script detects SSH and switches to manual mode.
- Option A: open a second terminal on your LOCAL machine and run: `ssh -L 17563:localhost:17563 user@` then visit the printed URL in your browser.
- Option B: visit the auth URL in any browser, authorise, then copy the redirect URL from the address bar (even if it shows a connection error) and paste it at the prompt.

Bot connects but !ask does nothing

- Check that the bot account is joined to the right channel (TWITCH_CHANNEL in .env).
- Verify the bot has not been banned or timed out in your channel.
- Check the terminal for [LLM] or [TTS] error messages.
- Ensure Ollama is running: `curl http://localhost:11434/api/tags`

No audio in OBS

- The Browser Source must have Control audio via OBS checked.
- Right-click the Browser Source → Properties → tick the checkbox.
- Check that OBS Browser audio track is not muted in the Audio Mixer.
- Test TTS manually: `echo 'test' | ./piper/piper --model ./piper/en_US-amy-medium.onnx --output_file /tmp/t.wav && aplay /tmp/t.wav`

[TTS] Piper binary not found

- Re-run `./install.sh` — it will re-download the Piper binary.
- Check `PIPER_BINARY` in `.env` points to `./piper/piper`.
- Verify: `ls -la ./piper/piper` (should be executable)
- Manually fix: `chmod +x ./piper/piper`

[LLM] Cannot connect to Ollama

- Start Ollama manually: `ollama serve`
- Check it is listening: `curl http://localhost:11434/api/tags`
- If `OLLAMA_URL` is changed from default, verify the new address.

OBS browser source shows blank / error

- Confirm StreamPet is running: `curl http://localhost:8765/health`
- Check `SERVER_PORT` in `.env` matches the URL in OBS.
- Try refreshing the browser source: right-click → Refresh.
- Disable any firewall rules blocking port 8765.

Viewer gets cooldown message immediately

- `COOLDOWN_SECONDS` applies per username, not globally.
- Cooldowns reset on server restart.
- Reduce `COOLDOWN_SECONDS` in `.env` if needed.

LLM responses are too slow

- Ensure Ollama is using the GPU: `ollama ps` — look for GPU column.
- Switch to a smaller model: `OLLAMA_MODEL=phi3:mini`
- Reduce `MAX_TOKENS` in `.env` (e.g. 80).
- Check GPU driver: `nvidia-smi` — if not found, install CUDA drivers.

TypeError: Bot.__init__() missing 'client_id', 'client_secret', 'bot_id'

- This means TwitchIO v3 was installed instead of v2. The APIs are incompatible.
- Fix: run `.venv/bin/pip install "twitchio>=2.10.0,<3.0.0"`
- `requirements.txt` already pins `twitchio<3.0.0` — this error only occurs if pip was run manually without the pin.
- Verify the version: `.venv/bin/pip show twitchio | grep Version` (should show 2.x.x)

get_token.py — 'invalid client' error in browser

- This means `TWITCH_CLIENT_ID` or `TWITCH_CLIENT_SECRET` in `.env` still contain placeholder values.
- Edit `.env` and replace the placeholder strings with your real credentials from dev.twitch.tv/console.
- The script checks for common placeholders (e.g. `'your_client_id_here'`) and will show a clear error if they are detected.

14. File Reference

Every file in the project and its purpose:

File	Purpose
<code>main.py</code>	Entry point. Runs FastAPI server and Twitch bot concurrently via <code>asyncio.gather</code> .
<code>server.py</code>	FastAPI app: serves browser source (GET /), WebSocket (/ws), audio files, and health check.
<code>bot.py</code>	TwitchIO bot. Handles !ask command, per-user cooldown, calls LLM and TTS, broadcasts to WebSocket.
<code>llm.py</code>	Async Ollama client. Builds system prompt from config, sends question, returns answer text.
<code>tts.py</code>	Piper TTS wrapper. Runs Piper as async subprocess, saves WAV to <code>static/audio/</code> , cleans up old files.
<code>config.py</code>	Centralised config. Reads all settings from <code>.env</code> via <code>python-dotenv</code> .
<code>.env</code>	Your personal configuration (Twitch credentials, model names, ports). Not committed to Git.
<code>.env.example</code>	Template for <code>.env</code> . Safe to commit. Copy to <code>.env</code> and fill in real values.
<code>requirements.txt</code>	Python package list: <code>fastapi</code> , <code>uvicorn</code> , <code>twitchio</code> (pinned <3.0), <code>httpx</code> , <code>python-dotenv</code> , <code>aiofiles</code> , <code>websockets</code> .
<code>install.sh</code>	One-shot installer: <code>pip</code> , <code>venv</code> , Python deps, Piper binary, voice model, Ollama, <code>llama3.2</code> .
<code>start.sh</code>	Daily start script. Activates <code>venv</code> , optionally starts Ollama daemon, runs <code>main.py</code> .
<code>get_token.py</code>	Self-contained Twitch OAuth helper. Reads Client ID/Secret from <code>.env</code> , opens browser for Twitch auth, captures the redirect on <code>localhost:17563</code> , and writes <code>TWITCH_TOKEN</code> back to <code>.env</code> . SSH-aware (offers paste or tunnel method). No extra packages needed.
<code>setup_service.sh</code>	Installs StreamPet as a persistent system-level systemd service. Run once with <code>sudo</code> after configuration is complete. Pre-flight checks token and channel before installing.
<code>remove_service.sh</code>	Stops, disables, and removes the systemd service unit. StreamPet files are left intact.
<code>make_placeholders.py</code>	Generates solid-colour placeholder <code>pet_idle.png</code> and <code>pet_talk.png</code> without any dependencies.
<code>static/index.html</code>	OBS browser source page. Contains <code>pet </code> and speech bubble <code><div></code> .
<code>static/style.css</code>	All CSS: bubble styling, pet image sizing, talking glow effect.
<code>static/pet.js</code>	WebSocket client: connects to <code>/ws</code> , handles thinking/speak/error events, swaps PNG frames, plays audio.

<code>static/pet_idle.png</code>	YOUR artwork — pet with mouth closed. Replace placeholder with your own PNG.
<code>static/pet_talk.png</code>	YOUR artwork — pet with mouth open. Alternates with idle during TTS playback.
<code>static/pet_placeholder.svg</code>	Fallback shown if <code>pet_idle.png</code> is missing (displayed via <code>onerror</code> in <code>index.html</code>).
<code>static/audio/</code>	Generated TTS WAV files. Auto-cleaned after 5 minutes.
<code>piper/piper</code>	Piper TTS binary (downloaded by <code>install.sh</code> from GitHub releases).
<code>piper/*.onnx</code>	Piper ONNX voice model files. Download additional voices from Hugging Face.
<code>CLAUDE.md</code>	Project context file read by Claude Code on folder open. Contains architecture, decisions, and task history.